Национална олимпиада по Информационни технологии 2025/2026
Проект №146 : Оксалейт

1. Тема: Интелигентна система за оптимизиране на търсенето в Интернет

Регистрационен номер на проект: 146
Направление: Разпределени приложения

2. Автори

Участник 1

Име: Симеон Николаев Вангелов
ЕГН : 0841250445
Адрес : гр. Бургас, кв. Банево, ул. Тракия 6
Тел : 0988903144
Имейл : simmeon.nv@proton.me
Училище: ППМГ „Акад. Никола Обрешков“ Бургас
Клас: 11

Участник 2

Име: Любомир Христов Иванов
ЕГН : 0846230462
Адрес : гр. Бургас, ул. "Сливница", № 31
Тел : 09885556450
Имейл : bvvn@protonmail.com
Училище: ППМГ „Акад. Никола Обрешков“ Бургас,
Клас: 11

3. Ръководител

Име: д-р Александър Иванов
Тел:. 0988378335
Електронна поща: alex.ivanov@bfu.bg
Длъжност: Старши учител в ППМГ „Акад. Никола Обрешков“ Бургас

4. Резюме

4.1. Цели

Целта на проекта е да предостави сигурна и иновативна платформа за откриване на информация в интернет, която с оглед на безопасността и анонимността на потребителите. Системата е проектирана да функционира, както като напълно самостоятелна търсачка, така и като мета-търсачка, която агрегира и оптимизира резултати от други търсещи машини. Уникална характеристика на проекта е предоставянето на сложни и интерактивни графови визуализации на заявките за търсене, което позволява на потребителите да изследват и осмислят връзките между различните данни по визуален начин.

Наличните функционалности, които ще се поддържат, са:

- Самостоятелно търсене на информация със собствено индексиране
- Мета-търсене, обединяващо и филтриращо резултати от външни търсачки
- Генериране на интерактивни графови визуализации въз основа на потребителската заявка
- Осигуряване на пълна анонимност на потребителя (без проследяване и запазване на лични данни)

По време на разработка на проекта се използва платформата Github, която е Git система за контрол и управление на версиите.

Програмният код на проекта е достъпен на следните адреси:

Главният код: https://github.com/simeonnv/Oxalate
Api gateway: https://github.com/simeonnv/Servo
Кодът на проекта е лицензиран под GNU General Public License v3.0
Приложението е достъпно на следния адрес: https://oxalate.ate6.org/

4.2. Основни етапи в реализирането:

- Анализ на различните технологии за реализация на проекта
- Планиране на архитектурата
- Програмна реализация
- Тестване на основните функционалности
- Преработка и отстраняване на бъгове
- Имплементация и визуализация на резултати
- Тестове и анализи

4.3. Ниво на сложност на проекта:

Проектът се характеризира с високо ниво на сложност, обусловено от съчетанието на архитектурни, алгоритмични и технологични предизвикателства, както и от необходимостта за осигуряване на висока производителност, сигурност и мащабируемост.

Една от основните трудности при реализацията е проектирането и синхронизирането на разпределена архитектура с микроуслуги, в която отделните компоненти функционират самостоятелно, но трябва да обменят данни надеждно и ефективно. Това изисква внимателно планиране на интерфейсите между услугите, обработката на грешки и контрола на достъпа.

Съществен проблем представлява и обработката на големи обеми от данни в реално време, включително асинхронен и паралелен уеб скрейпинг, индексиране и търсене. За

решаването му са използвани неблокиращи структури от данни и конкурентни алгоритми, с цел предотвратяване на блокиране на системата и загуба на производителност.

Допълнителна сложност произтича от използването на различни типове бази данни, всяка с различен модел на съхранение и предназначение – релационна, графова, кешираща и търсеща. Това налага синхронизиране на данните между тях и оптимизиране на заявките според спецификата на всяка база. Проектът включва и реализация на механизми за сигурност, като автентикация и авторизация чрез JWT токени с асиметрична криптография (RSA).

4.4. Логическо и функционално описание на решението

Системата предлага следния набор от функции за крайните потребители и администраторите:

- Търсене и филтриране: Възможност за извършване на комплексни заявки с висока скорост и точност.
- Интерактивна граф визуализация: Потребителският интерфейс предоставя динамично платно (canvas), където резултатите са представени като свързани възли. Потребителите могат да взаимодействат с графа (мащабиране, местене, кликване върху възли за детайли).
- Мета-търсене: Интеграция с външни търсачки с цел обединяване на резултати от различни платформи в единен визуален формат.
- Административен панел (admin _ui): За управления на скрейпинг механизма.

4.5. Реализация

4.5.1. Обща архитектура

При реализацията на проекта са използвани съвременни технологии, съобразени с изискванията за висока производителност, мащабируемост и сигурност:

Избор на технологични средства

Програмни езици

- Rust - използван за backend компонентите поради високата си производителност, безопасност на паметта и добри възможности за паралелна обработка [1].
- Typescript - използван за frontend частта с цел по-добра типова сигурност и поддържаемост на кода [2].
- Nix - използван за автоматизиране на средата за разработка и изграждане на Docker изображения [3].

Платформи и библиотеки

- NuxtJS – за разработка на frontend интерфейса [4].
- Axum – backend framework за Rust, ориентиран към изграждане на REST API [5].
- Utoipa и Swagger – за автоматично генериране на API документация [6][7].

Бази данни

- Redis - използва се като бърз кеш за често достъпвани страници [8].
- Postgres - основна релационна база данни с разширени възможности за пълнотекстово търсене и алгоритъм BM25 за класиране на резултатите [9].
- Neo4j - графова база данни, използвана за съхраняване и визуализиране на релациите между думи и страници [10].

- Elastic Search - използва се за оптимизирано търсене и обработка на телеметрични данни [10].

Инструменти за деплоймънт

- Docker - емулация на вируални машини
- Kubernetes - оркестратор за Docker инстанции

Архитектура и микроуслуги

Проектът е реализиран като разпределена система с микроуслуги. Основните компоненти са:

- **servo** - обратен прокси сървър, което играе ролата на "API gateway". Всички заявки минават през него. Той осигурява това, че потребителя е авторизиран и забързва заявки като връща предварително готови заявки от кеша (Redis).
- **servo_auth** - авторизатор. Капсулира криптографичните подписни ключове нужни за създаването на нови refresh токени и session токени.
- **oxalate_harvester** - оркестратор за oxalate_outlet инстанциите. Обработва страници и ги запазва в базите от данни.
- **oxalate_outlet** - отговаря за скрейпването на страници. Праща всички заявки към oxalate_harvester за обработка.
- **oxalate_indexer** – използва се за обработката към заявката за търсене на потребителя и съответно индексира информацията от базите от данни.
- **frontend** – потребителски интерфейс.
- **admin_ui** – административен интерфейс.
- **kafka_log_visualizer** – интефейс за debugging. От него е възможен достъп и филтрация на всички логове в реално време.

4.5.2. Организация на файловете

Файловете flake.nix и flake.lock са позиционирани е главната папка. Те се ползват за създаване средата за програмиране, нужна за работа по кода.

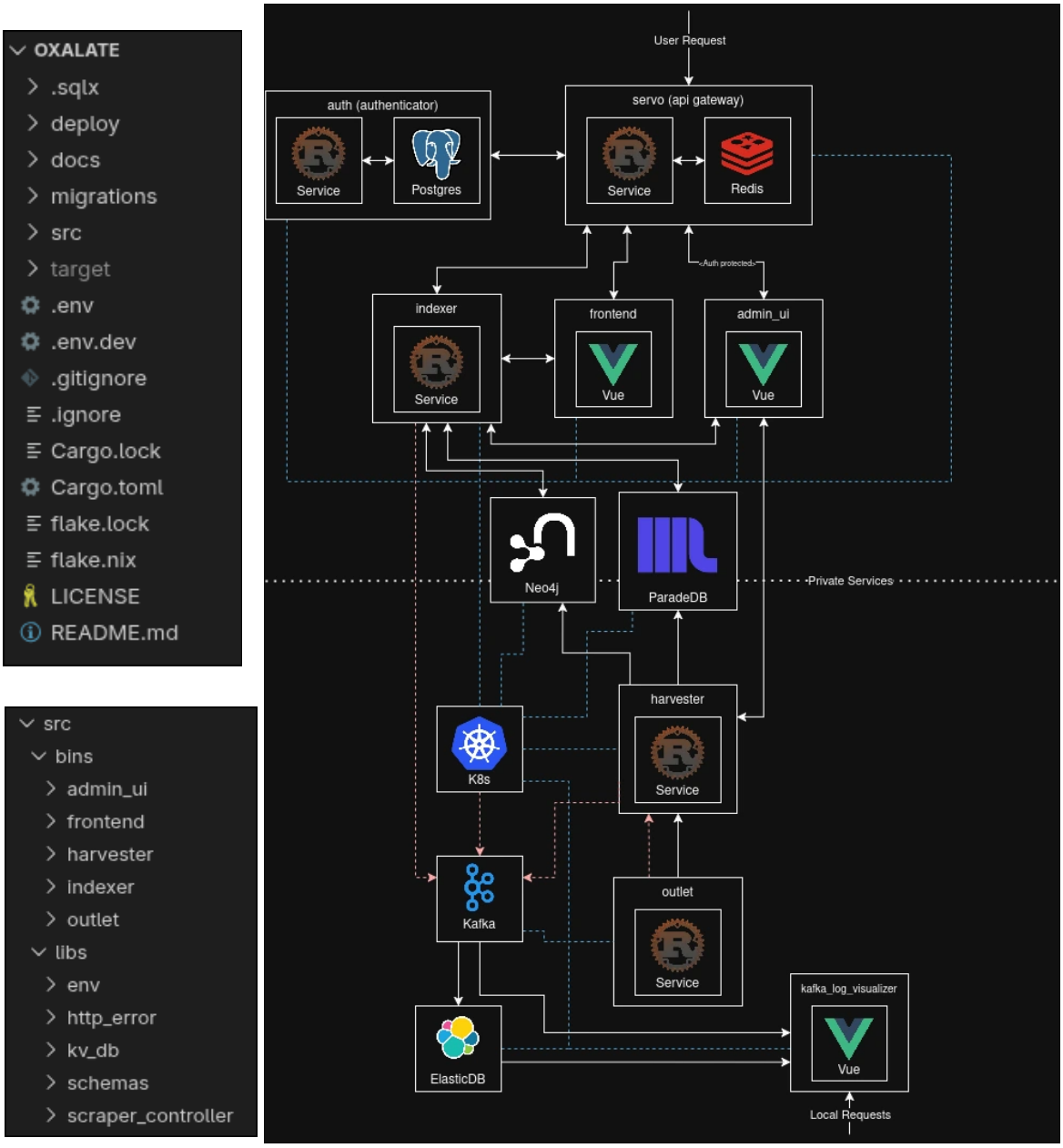
В папката migrations/ са разположени всички SQL заявки за инстанциране на таблиците от Postgres базата данни.

В папката deploy/ е кодът за деплойване.

Документацията на проекта е позиционирана в папката в docs.

И в src е разположен главният код.

На Фигури 1 а) и б) са представени логически описания на структурата и технологиите на проекта.



Фиг.1. а) Структура на проекта **б)** Взаимовръзки между изпозлваните технологии

4.5.3. Подредба на кода в бизнес логиката

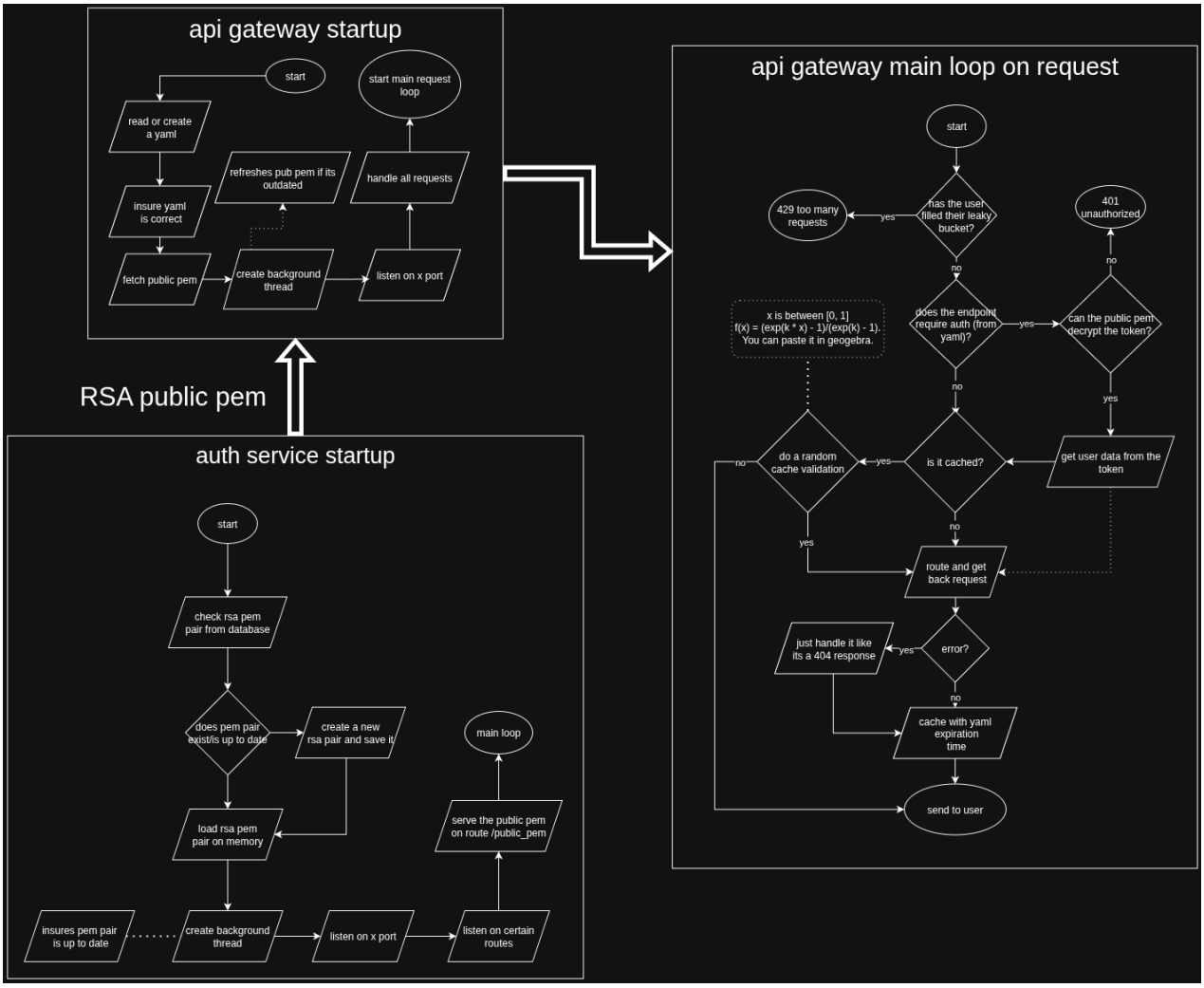
- src/ папката е разделена на 2 части:
- bins/, където е разположен кодът на всяка микроуслуга.
  libs/ , съдържаща споделения код на всички микроуслуги.
- libs/env съдържа кода, отговарящ за четенето и десериализирането на env variables.
- libs/http_error съдържа структура от данни, която е маркиран съюз (tagged union), който се ползва за представянето на всички типове HTTP кодови грешки.
- libs/kv_db съдържа код за система за запазване и синхронизиране на програмното състояние (state).-
- libs/schemas е предназначена за дефиниции на структурите за заявки (Req) и отговори (Res) на всички API endpoints на всички сервиси
- libs/scraper_controller съдържа системата за генериране и оркестриране на задачи за скрейпърите.

4.5.4. Дизайн на кода

Проектът е реализиран като високопроизводително REST API на Rust, което съчетава функционален и императивен стил с централизирано състояние, използвайки асинхронни неблокиращи структури (DashMap), атомарни променливи и ARC указатели за гарантиране на безопасност при паралелна работа и прецизна документация чрез Swagger.

4.5.5. Авторизация

Реализацията на системата се основава на високопроизводителна архитектура, която съчетава функционален и императивен стил на програмиране в езика Rust за постигане на максимална предвидимост и сигурност на данните чрез централизирано управление на състоянието. Паралелизмът в приложението е обезпечен чрез използването на неблокиращи структури от данни и атомарни указатели с брояч на референциите, което предотвратява забавяния при четене и елиминира рисковете от надпревара на нишките. Комуникацията се осъществява чрез REST API интерфейс със JSON сериализация, придружен от автоматично генерирана документация чрез инструментите на Swagger за по-лесно интегриране и поддръжка. Сигурността е гарантирана от многослоен механизъм за автентикация и оторизация, базиран на JSON Web Tokens и дълготрайни рефреш токени, които позволяват автоматично подновяване на 15-минутни сесии без повторно въвеждане на парола. За криптографска защита на тези токени се прилага асиметричният алгоритъм RSA, при който частният ключ служи за подписване на достъпа, а публичният за неговата верификация, осигурявайки надеждна бариера пред нерегламентирани опити за достъп до административния интерфейс и критичните ресурси на системата. (Фиг.2.)



Фиг.2. Логическа схема на процеса на авторизация

Авторизацията се върши от 2 : auth и servo (api gateway).

- Auth service

Auth сервиса играе роля на енкапсулатор. Той отговаря за регенерирането на RSA ключовете и създаването на JWT. Това за Auth услугата не отговаря е за верификацията на JWT. Auth услугата има точка ендпоинт /public_pem, който връща RSA публичния ключ с целта всеки сервис да може да провери дали е валиден JWT.

- Servo

Servo взима публичния ключ от auth сервиса и с него проверява дали потребителят ползва валиден JWT.

4.5.6. Скрейпинг

Скрейпингът се изпълнява от 2 услуги в екип: oxalate_outlet и oxalate_harvester.

- Oxalate_harvester

Той е оркестраторът на всички oxalate_outlet инстанции. Той генерира “задачи”, които се пращат на всички тях. Една задача се състои от списък с домейни, използвания от тях

протокол, методи за протокола и headers. Инстанциите на oxalate_outlet връщат “решенията” на тези задачи, които представляват суровия HTML, върнат от всеки домейн. От този суров HTML рекурсивно се извлича текстът от всички етикети, като се премахват ненужните (например <script>, <style>, <svg>). От суровия текст се премахват всички препинателни знаци и съюзи. Накрая информацията се обработва и изпраща към съответната база данни.

- Oxalate_outlet

Услугата приема “задачи” от oxalate_harvester. При получаване на заявка, тя започва асинхронно и паралелно изпълнение на заявки към всеки домейн в списъка, използвайки буфериране. След приключване на обработката, “решението” се връща към oxalate_harvester, след което се приема следващата задача.

Тъй като услугата не съхранява собствено състояние (stateless), тя зависи изцяло от входящите данни от oxalate_harvester. Това я прави изключително ефективна за хоризонтално мащабиране. Единственото ограничение за вертикално мащабиране на oxalate_outlet е лимитът на TCP сокетите на системното ядро (kernel). Ако лимитът се махне, ограничението става самото системно ядро.

4.5.7. Бази от данни

За една търсачка е много важно как се запазва и структурира информация, за да връща резултати бързо. За тази цел са ползват 2 бази данни.

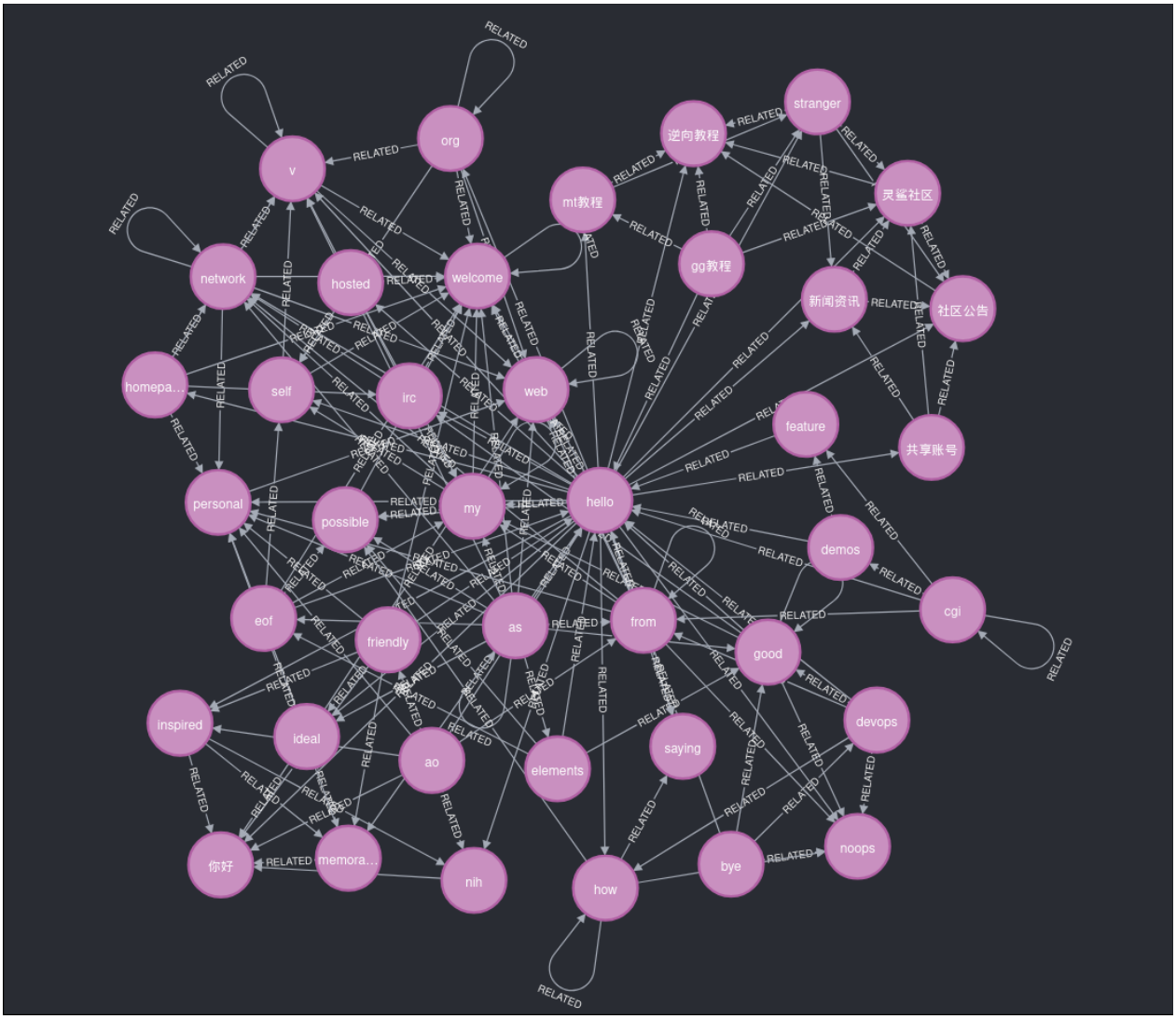
- ParadeDB (Postgres)

Това е модифицирана версия на postgres с векторни способности, пълнотекстово търсене и токенизатори. ParadeDB се използва за пълнотекстово търсене в голямата база от данни, като интегрира сложни търсещи алгоритми в Postgres. Основното предимство тук е използването на алгоритъма BM25, който служи за прецизно класиране (ranking) на резултатите според тяхната релевантност. BM25 оценява значението на ключовите думи спрямо общата дължина на текста и честотата на срещането им.

- Neo4J

За оптимизирана визуализация на връзките в резултатите от търсенето се използва графовата релационна база данни Neo4j. Процесът на изграждане на графа преминава през следните етапи: Приложението е разбито на четири основни групи компоненти, като някои от тях са разделени и на по-малки, за извършване на една конкретна функционалност.

1. Обработка на съдържанието: След като компонентът oxalate_harvester обработи страница с ключови думи, той прилага алгоритъм тип “плъзгащ се прозорец” (sliding window) върху извлечените термини.
2. Създаване на релации: За всяка комбинация от думи, попадащи в рамките на прозореца, се дефинира релация (връзка) в базата данни.
3. Изчисляване на тежестта: Ако дадена релация вече съществува, нейната “тежест” (weight) се инкрементира.
4. Визуализация: Финалните визуализации чрез граф се генерират динамично въз основа на изчислената тежест на релациите и общата честота на срещане на ключовите думи. (Фиг. 3)



Фиг.3. Визуалиация на резултати от търсене

4.6. Описание на приложението

Цялата среда за разработка е автоматизирана с помощта на Nix и Docker Compose. Необходимо е да се следват тези стъпки, за да се подготви локалната машина:

- Стартиране на инфраструктурата

  Първо трябва да вдигнете необходимите услуги (бази данни и др.) чрез Docker. Отидете в директорията за деплоймънт и стартирайте контейнерите:

  cd deploy/dev

  docker-compose up –d

- Влизане в работната среда

  Вместо да инсталирате зависимости ръчно, използвайте Nix, за да влезете в изолирана среда с всички нужни инструменти:

nix develop

- Изпълнение на миграции

  За да синхронизирате схемата на базата данни, използвайте sqlx-cli. Изпълнете миграциите за целия workspace:

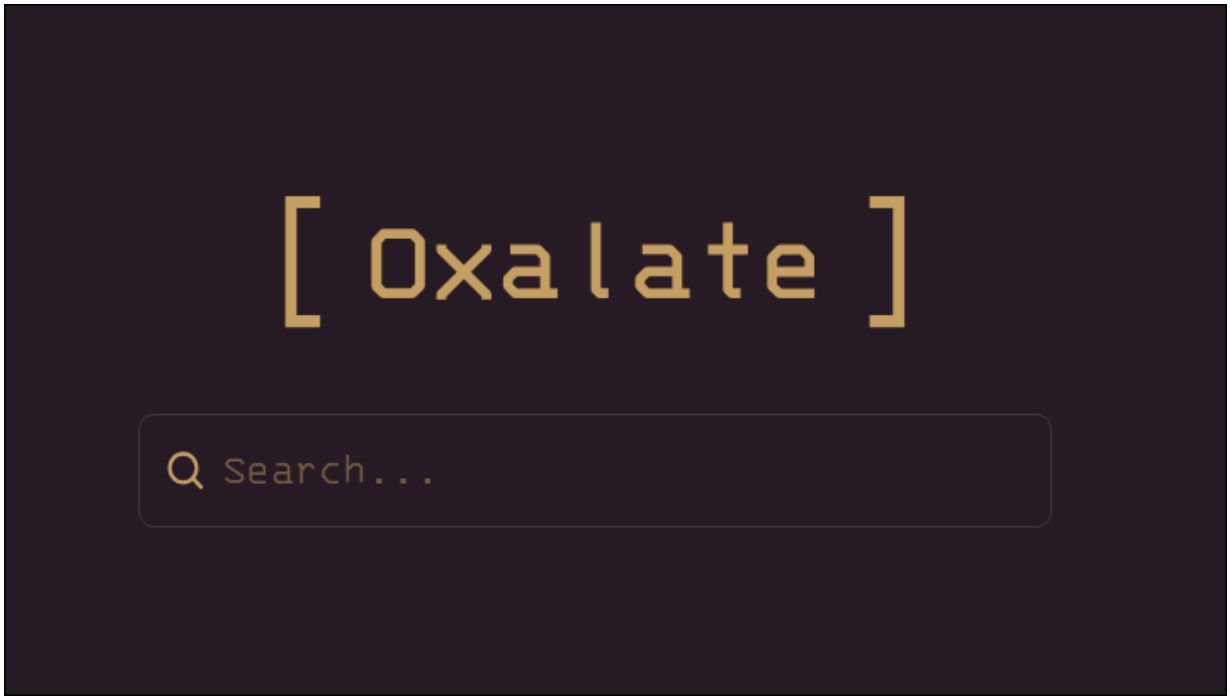
  sqlx migrate run –workspace

  https://oxalate.ate6.org/admin

  Username: NOIT

  Password: noit1234

На Фиг. 4. е показана екранна снимка на основния терфейс



Фиг.4. Основен интерфейс

4.7. Заключение

В резултат на извършеното проучване, проектиране и реализация беше създадена работеща интелигентна система за оптимизиране на търсенето в интернет, която обединява сигурност, висока производителност и иновативна визуализация на резултатите. Проектът успешно реализира поставените цели, като предоставя както стандартно търсене на информация, така и интерактивно представяне на връзките между данните чрез графови структури.

Разработеното приложение е функционално и стабилно, като към момента може да бъде използвано от реални потребители чрез уеб интерфейс. Архитектурата на системата е проектирана модулно и мащабируемо, което позволява лесно разширяване и поддръжка на

кода, както и добавяне на нови функционалности без съществени промени в съществуващата логика.

Съществуват редица възможности за бъдещо усъвършенстване на проекта. Сред тях са:

- внедряване на по-усъвършенствани алгоритми за семантично търсене и анализ на съдържанието;
- използване на машинно обучение за по-прецизно класиране и персонализиране на резултатите;
- оптимизация на графовите визуализации при работа с по-големи обеми от данни;
- разширяване на поддържаните източници за мета-търсене;
- добавяне на допълнителни административни инструменти за мониторинг и анализ на натоварването.

Проектът представлява солидна основа за бъдещо развитие и надграждане.

Източници

1. https://doc.rust-lang.org/stable/

2. https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html

3. https://nixos.org/

4. https://nuxt.com/docs/4.x/getting-started/introduction

5. https://docs.rs/axum/latest/axum/

6. https://docs.rs/utoipa/latest/utoipa/

7. https://swagger.io/

8. https://redis.io/

9. https://www.postgresql.org/docs/

10. https://neo4j.com/